Logo — Reference

Logo is a turtle-graphics interpreter (written in our C, compiled to .NET IL by `cc`). It uses an *arity-directed* reader (you must know each word's arity to parse its arguments, so there is no yacc grammar). A turtle moves under your commands; the path renders to **SVG, PNG, or an animated GIF** — or to a live window — and `PRINT` writes text.

```
logo prog.logo -png out.png      # render the drawing to PNG  (also -svg, -gif)
logo prog.logo                   # run for PRINT/text output
logo                             # interactive REPL (live window via the gfx host)
```

Quick Reference

```
FD 100  BK 50  RT 90  LT 45        ' move / turn (FORWARD BACK RIGHT LEFT)
PU  PD  HOME  CS                   ' pen up/down, home, clear screen
SETPC 4   SETPENSIZE 3   SETXY x y   SETH deg
REPEAT 4 [ FD 100 RT 90 ]          ' repeat a command list (:repcount = current index)
IF cond [ ... ]    IFELSE cond [ ... ] [ ... ]
TO name :a :b ... END              ' define a procedure;  OUTPUT v / STOP
MAKE "x 10        :x               ' set a variable / read it
SUM DIFFERENCE PRODUCT  RANDOM SQRT SIN COS  + - * /     PRINT  SHOW
```

Data types

Numbers, **words** (`"hello` — a quoted word), and **lists** (`[a b c]`, also used for command blocks). Variables hold any of these. Booleans are produced by comparisons for `IF` / `IFELSE`.

Statements / Commands

Turtle motion (`FD` / `BK` / `RT` / `LT` / `SETXY` / `SETH` / `HOME`), pen (`PU` / `PD` / `SETPC` / `SETPENSIZE` / `CS`), control (`REPEAT n [ ... ]`, `IF c [ ... ]`, `IFELSE c [ ... ] [ ... ]`), definitions (`TO … END`), variables (`MAKE`), and output (`PRINT` / `SHOW`).

Functions

Procedures are `TO name :params … END`; a procedure that `OUTPUT`s a value is an *operation* usable inside expressions, while one that just acts is a *command*. Built-in operations: `SUM DIFFERENCE PRODUCT QUOTIENT REMAINDER POWER RANDOM SQRT SIN COS INT ABS` (plus infix `+ - * /`). Procedures may recurse (Activity 5).

Input / Output

`PRINT` writes a value/list and a newline; `SHOW` is similar. Turtle output goes to a drawing: pass `-png file`, `-svg file`, or `-gif file` (animated, capturing a frame per move), or run with no file argument for a live window.

Graphics

This is the point of Logo. The turtle starts at the centre facing up; `FD` / `RT` etc. draw lines in the current pen colour (`SETPC 0..15`) and width (`SETPENSIZE`). The path is recorded so it can be exported to SVG/PNG/GIF.

Notes

- Arity-directed reader: each command consumes exactly as many arguments as its arity, so there are no statement separators.
- Headless rendering (`-png` / `-svg` / `-gif`) needs no window; `-gif` makes an animation of the turtle drawing. The live REPL needs the windowed gfx host.
- `:repcount` gives the current `REPEAT` index (1-based).

Subset boundaries

A focused turtle Logo. Not included: arrays, full list processing (`FIRST` / `BUTFIRST` / `FPUT`), property lists, `RUN` / `parse`, dynamic word/ `THING`, and most of the library of a full Logo. Variables and procedures are the state.

Tutorial

Every example was run with `logo`; text output is real and graphics were rendered to PNG and viewed.

1. Your first program

```
SETPENSIZE 3
SETPC 1
REPEAT 4 [ FD 150 RT 90 ]
```

```
logo square.logo -png square.png
```

Renders a **blue square**: the turtle goes forward 150 and turns right 90°, four times, back to where it started. (`SETPC 1` = blue; `SETPENSIZE 3` thickens the pen.)

2. Variables and data types

```
PRINT [Hello from Logo]
MAKE "n 6
PRINT :n * :n
```

```
Hello from Logo
36
```

`[Hello from Logo]` is a list; `MAKE "n 6` sets the variable `n`; `:n` reads it. Values are numbers, words, or lists.

3. Flow control

`REPEAT` runs a command list a fixed number of times (it is how Activity 1's square is drawn); `IF` / `IFELSE` branch on a condition. The variable `:repcount` is the current repetition (1-based):

```
REPEAT 3 [ IFELSE :repcount < 3 [ PRINT :repcount ] [ PRINT [last] ] ]
```

```
1
2
last
```

`IFELSE` picks one of two command lists depending on the test.

4. Arrays

Logo here has no array type (see *Subset boundaries*) — iteration is by `REPEAT` over a count or by recursion over a shrinking value. The spirograph in Activity 8 builds a 36-element rosette purely by `REPEAT 36 [ ... ]`, never storing an array.

5. Subroutines and functions

`TO … END` defines a procedure; procedures take `:parameters` and may **recurse**:

```
TO spiral :len :angle
  IF :len < 3 [ STOP ]
  FD :len
  RT :angle
  spiral :len - 4 :angle
END
SETPENSIZE 2
SETPC 4
spiral 120 28
```

```
logo spiral.logo -png spiral.png
```

Renders a **red spiral**: `spiral` draws one segment, turns, then calls itself with a shorter `:len` until `STOP` ends the recursion.

6. Memory management

There is no manual memory — Logo manages variables and turtle state:

- `MAKE "x v` creates/updates a variable; `:x` reads it; procedure `:params` are locals for that call.
- **Turtle state** (position, heading, pen) is updated by each command, and every line drawn is **recorded** so the path can be exported to SVG/PNG/GIF afterward.
- Nothing is freed by hand.

7. Strings and a text layout

Text is words and lists, printed with `PRINT` / `SHOW`:

```
MAKE "name [Ada Lovelace]
PRINT [Hello]
PRINT :name
```

```
Hello
Ada Lovelace
```

Words and lists are the text medium — there are no string-manipulation functions in this subset (see *Subset boundaries*), so you lay text out by printing words and lists.

8. Drawing a picture

Drawing is native. This program layers a red five-point star over a blue rosette of 36 rotated squares:

```
TO square :size
  REPEAT 4 [ FD :size RT 90 ]
END
TO star
  REPEAT 5 [ FD 150 RT 144 ]
END
SETPENSIZE 2
SETPC 4
star
PU HOME PD
SETPC 1
REPEAT 36 [ square 80 RT 10 ]
PRINT [done]
```

```
logo art.logo -png art.png      ' (PRINT also writes:)  done
```

Rendered, this is a red star inside a blue spirograph ring — `star` walks a 144° turn five times; the `REPEAT 36 [ square 80 RT 10 ]` draws 36 squares each rotated 10°. Render with `-gif` instead to watch the turtle draw it frame by frame.

9. Where to go next

Logo's payoff is pictures: render with `-svg` (vector), `-png` (raster), or `-gif` (animation), or run with no file for a live window. Build procedures that recurse for spirals and trees, drive colour with `SETPC`, and combine `REPEAT` with `RT` for rosettes. Fuller list processing and arrays are the natural extensions (*Subset boundaries*).